

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Интеллектуальный анализ данных»

Тема: «Автоэнкодеры»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е.А.

Проверил:

Никитин С. К.

Цель работы: Научиться применять автоэнкодеры для осуществления визуализации данных и их анализа

Ход работы:

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент с использованием нейросетевой модели автоэнкодера (с двумя и тремя нейронами в среднем слое);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Реализовать метод t-SNE для визуализации данных (использовать также 2 и 3 компонента), построить соответствующую визуализацию;
4. Применить к данным метод PCA (2 и 3 компонента), реализованный в ЛР №1, сделать выводы;
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№	Выборка	Класс
8	Rice (Cammeo and Osmancik)	Class

Код программы:

```
import time
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from scipy.io import arff
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.manifold import TSNE
from sklearn.neighbors import NearestNeighbors

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

RAW_PATH = r"D:\rice+cammeo+and+osmancik\Rice_Cammeo_Osmancik.arff"

def load_data(path: str) -> pd.DataFrame:
    data, _meta = arff.loadarff(path)
    df = pd.DataFrame(data)
    if isinstance(df["Class"].iloc[0], (bytes, bytearray)):
        df["Class"] = df["Class"].str.decode("utf-8")
    return df

df = load_data(RAW_PATH)
```

```

FEATURE_COLS = [
    "Area", "Perimeter", "Major_Axis_Length", "Minor_Axis_Length",
    "Eccentricity", "Convex_Area", "Extent",
]
X_raw = df[FEATURE_COLS].values.astype(float)
y = np.asarray(df["Class"], dtype=object)

print("Пропуски по признакам (до замещения):")
print(pd.DataFrame(X_raw, columns=FEATURE_COLS).isna().sum().to_string())

imputer = SimpleImputer(strategy="median")
X_imputed = imputer.fit_transform(X_raw)

print(f"\nРазмер выборки после замещения пропусков: {X_imputed.shape[0]} "
      f"объектов, {X_imputed.shape[1]} признаков")
print(df["Class"].value_counts().to_string())
print()

scaler = StandardScaler()
X = scaler.fit_transform(X_imputed)

classes = ["Cammeo", "Osmancik"]
colors = {"Cammeo": "tab:blue", "Osmancik": "tab:orange"}
markers = {"Cammeo": "o", "Osmancik": "^"}

def scatter_2d(ax, data, labels, title, xlabel="Компонента 1", ylabel="Компонента 2"):
    for c in classes:
        mask = labels == c
        ax.scatter(data[mask, 0], data[mask, 1], label=c,
                   color=colors[c], marker=markers[c],
                   alpha=0.55, edgecolor="k", linewidths=0.3, s=25)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.legend()
    ax.grid(alpha=0.3)

def scatter_3d(ax, data, labels, title,
               xlabel="Компонента 1", ylabel="Компонента 2", zlabel="Компонента 3"):
    for c in classes:
        mask = labels == c
        ax.scatter(data[mask, 0], data[mask, 1], data[mask, 2], label=c,
                   color=colors[c], marker=markers[c],
                   alpha=0.55, edgecolor="k", linewidths=0.3, s=25)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_zlabel(zlabel)
    ax.set_title(title)
    ax.legend()

def pca_manual(data: np.ndarray, n_components: int):
    cov_matrix = np.cov(data.T)
    eigvals, eigvecs = np.linalg.eig(cov_matrix)
    eigvals = eigvals.real
    eigvecs = eigvecs.real

    order = np.argsort(-eigvals)
    eigvals_sorted = eigvals[order]
    eigvecs_sorted = eigvecs[:, order]

```

```

W = eigvecs_sorted[:, :n_components]
projected = data.dot(W)
return projected, eigvals_sorted, eigvecs_sorted

X_pca_manual_2d, eigvals_manual, eigvecs_manual = pca_manual(X, 2)
X_pca_manual_3d, _, _ = pca_manual(X, 3)

pca2 = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca_2d = pca2.fit_transform(X)

pca3 = PCA(n_components=3, random_state=RANDOM_STATE)
X_pca_3d = pca3.fit_transform(X)

full_info = eigvals_manual.sum()
loss_2 = 100 - eigvals_manual[:2].sum() / full_info * 100
loss_3 = 100 - eigvals_manual[:3].sum() / full_info * 100

print("=" * 70)
print("PCA: собственные значения ковариационной матрицы (по убыванию)")
for i, v in enumerate(eigvals_manual, 1):
    print(f"  lambda_{i} = {v:.4f}    доля объясненной дисперсии = "
          f"{v / full_info * 100:5.2f}%")
print(f"\nПотери информативности при сведении к 2 компонентам: {loss_2:.2f}%")
print(f"Потери информативности при сведении к 3 компонентам: {loss_3:.2f}%")
print("Проверка через sklearn:")
print("  2 компоненты:", np.round(pca2.explained_variance_ratio_, 4),
      "  сумма =", round(pca2.explained_variance_ratio_.sum() * 100, 2), "%")
print("  3 компоненты:", np.round(pca3.explained_variance_ratio_, 4),
      "  сумма =", round(pca3.explained_variance_ratio_.sum() * 100, 2), "%")

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
scatter_2d(axes[0], X_pca_manual_2d, y, "PCA вручную", "PC1", "PC2")
scatter_2d(axes[1], X_pca_2d, y, "PCA через sklearn", "PC1", "PC2")
plt.suptitle("PCA: проекция на первые 2 главные компоненты", fontsize=13)
plt.tight_layout()
plt.savefig("pca_2d.png", dpi=150)
plt.show()
plt.close()

fig = plt.figure(figsize=(13, 6))
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
scatter_3d(ax1, X_pca_manual_3d, y, "PCA вручную")
ax2 = fig.add_subplot(1, 2, 2, projection="3d")
scatter_3d(ax2, X_pca_3d, y, "PCA через sklearn")
plt.suptitle("PCA: проекция на первые 3 главные компоненты", fontsize=13)
plt.tight_layout()
plt.savefig("pca_3d.png", dpi=150)
plt.show()
plt.close()

class Autoencoder:
    def __init__(self, layer_sizes, activations, bottleneck_idx, seed=RANDOM_STATE):
        self.layer_sizes = layer_sizes
        self.activations = activations
        self.bottleneck_idx = bottleneck_idx
        rng = np.random.default_rng(seed)
        self.W, self.b = [], []
        for i in range(len(layer_sizes) - 1):
            fan_in, fan_out = layer_sizes[i], layer_sizes[i + 1]

```

```

        std = np.sqrt(2.0 / fan_in) if activations[i] == "relu" else np.sqrt(1.0 / fan_in)
        self.W.append(rng.normal(0, std, size=(fan_in, fan_out)))
        self.b.append(np.zeros(fan_out))

    self.mW = [np.zeros_like(w) for w in self.W]
    self.vW = [np.zeros_like(w) for w in self.W]
    self.mb = [np.zeros_like(bb) for bb in self.b]
    self.vb = [np.zeros_like(bb) for bb in self.b]
    self.t = 0

    @staticmethod
    def _act(z, kind):
        return np.maximum(0, z) if kind == "relu" else z

    @staticmethod
    def _act_grad(z, kind):
        return (z > 0).astype(z.dtype) if kind == "relu" else np.ones_like(z)

    def forward(self, X):
        activations_list, zs = [X], []
        a = X
        for i in range(len(self.W)):
            z = a @ self.W[i] + self.b[i]
            a = self._act(z, self.activations[i])
            zs.append(z)
            activations_list.append(a)
        return activations_list, zs

    def encode(self, X):
        activations_list, _ = self.forward(X)
        return activations_list[self.bottleneck_idx]

    def train(self, X, epochs=600, batch_size=64, lr=1e-3,
              beta1=0.9, beta2=0.999, eps=1e-8, verbose_every=100, seed=0):
        n = X.shape[0]
        rng = np.random.default_rng(seed)
        history = []
        for epoch in range(1, epochs + 1):
            perm = rng.permutation(n)
            X_shuffled = X[perm]
            epoch_loss = 0.0
            for start in range(0, n, batch_size):
                batch = X_shuffled[start:start + batch_size]
                m = batch.shape[0]
                activations_list, zs = self.forward(batch)
                output = activations_list[-1]
                diff = output - batch
                epoch_loss += np.mean(diff ** 2) * m

            dA = 2 * diff / m
            dWs = [None] * len(self.W)
            dbs = [None] * len(self.b)
            for i in reversed(range(len(self.W))):
                dZ = dA * self._act_grad(zs[i], self.activations[i])
                dWs[i] = activations_list[i].T @ dZ
                dbs[i] = dZ.sum(axis=0)
                dA = dZ @ self.W[i].T

            self.t += 1
            for i in range(len(self.W)):

```

```

        self.mW[i] = beta1 * self.mW[i] + (1 - beta1) * dWs[i]
        self.vW[i] = beta2 * self.vW[i] + (1 - beta2) * (dWs[i] ** 2)
        mW_hat = self.mW[i] / (1 - beta1 ** self.t)
        vW_hat = self.vW[i] / (1 - beta2 ** self.t)
        self.W[i] -= lr * mW_hat / (np.sqrt(vW_hat) + eps)

        self.mb[i] = beta1 * self.mb[i] + (1 - beta1) * dbs[i]
        self.vb[i] = beta2 * self.vb[i] + (1 - beta2) * (dbs[i] ** 2)
        mb_hat = self.mb[i] / (1 - beta1 ** self.t)
        vb_hat = self.vb[i] / (1 - beta2 ** self.t)
        self.b[i] -= lr * mb_hat / (np.sqrt(vb_hat) + eps)

    epoch_loss /= n
    history.append(epoch_loss)
    if verbose_every and epoch % verbose_every == 0:
        print(f"    эпоха {epoch:4d}/{epochs}    MSE(восст.) = {epoch_loss:.5f}")
    return history

print("\n" + "=" * 70)
print("АВТОЭНКODEP: обучение модели с 2 нейронами в среднем слое")
t0 = time.time()
ae2 = Autoencoder(layer_sizes=[7, 16, 8, 2, 8, 16, 7],
                  activations=["relu", "relu", "linear", "relu", "relu", "linear"],
                  bottleneck_idx=3, seed=RANDOM_STATE)
hist_ae2 = ae2.train(X, epochs=600, batch_size=64, lr=1e-3, seed=1)
X_ae_2d = ae2.encode(X)
print(f"    обучение заняло {time.time() - t0:.1f} с, "
      f"итоговая MSE восстановления = {hist_ae2[-1]:.5f}")

print("\nАВТОЭНКODEP: обучение модели с 3 нейронами в среднем слое")
t0 = time.time()
ae3 = Autoencoder(layer_sizes=[7, 16, 8, 3, 8, 16, 7],
                  activations=["relu", "relu", "linear", "relu", "relu", "linear"],
                  bottleneck_idx=3, seed=RANDOM_STATE)
hist_ae3 = ae3.train(X, epochs=600, batch_size=64, lr=1e-3, seed=2)
X_ae_3d = ae3.encode(X)
print(f"    обучение заняло {time.time() - t0:.1f} с, "
      f"итоговая MSE восстановления = {hist_ae3[-1]:.5f}")

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(hist_ae2, label="узкий слой: 2 нейрона")
ax.plot(hist_ae3, label="узкий слой: 3 нейрона")
ax.set_xlabel("Эпоха")
ax.set_ylabel("MSE восстановления")
ax.set_title("Автоэнкодер: сходимость обучения")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("autoencoder_training_curve.png", dpi=150)
plt.show()
plt.close()

fig, ax = plt.subplots(figsize=(7, 6))
scatter_2d(ax, X_ae_2d, y, "Автоэнкодер (2 нейрона в среднем слое)")
plt.tight_layout()
plt.savefig("autoencoder_2d.png", dpi=150)
plt.show()
plt.close()

fig = plt.figure(figsize=(7.5, 6.5))

```

```

ax = fig.add_subplot(111, projection="3d")
scatter_3d(ax, X_ae_3d, y, "Автоэнкодер (3 нейрона в среднем слое)")
plt.tight_layout()
plt.savefig("autoencoder_3d.png", dpi=150)
plt.show()
plt.close()

print("\n" + "=" * 70)
print("t-SNE: подбор perplexity (2 компоненты)")
perplexities = [20, 30, 40, 50, 60]
tsne_results_2d = {}
for p in perplexities:
    t0 = time.time()
    emb = TSNE(n_components=2, perplexity=p, init="pca",
               learning_rate="auto", random_state=RANDOM_STATE).fit_transform(X)
    tsne_results_2d[p] = emb
    print(f" perplexity={p:2d} ({time.time() - t0:5.1f} c)")

fig, axes = plt.subplots(1, len(perplexities), figsize=(4.2 * len(perplexities), 4.6))
for ax, p in zip(axes, perplexities):
    scatter_2d(ax, tsne_results_2d[p], y, f"perplexity={p}", "t-SNE 1", "t-SNE 2")
    ax.get_legend().remove()
axes[0].legend(loc="upper left", fontsize=8)
plt.suptitle("t-SNE (2D): сравнение значений perplexity", fontsize=13)
plt.tight_layout()
plt.savefig("tsne_2d_perplexity_comparison.png", dpi=150)
plt.show()
plt.close()

BEST_PERPLEXITY = 30
X_tsne_2d = tsne_results_2d[BEST_PERPLEXITY]

fig, ax = plt.subplots(figsize=(7, 6))
scatter_2d(ax, X_tsne_2d, y, f"t-SNE (2D), perplexity={BEST_PERPLEXITY}",
           "t-SNE 1", "t-SNE 2")
plt.tight_layout()
plt.savefig("tsne_2d_best.png", dpi=150)
plt.show()
plt.close()

print(f"\nВыбранное значение perplexity для итоговой визуализации: {BEST_PERPLEXITY}")

print("\nt-SNE: построение 3-компонентной проекции "
      f"(perplexity={BEST_PERPLEXITY})")
t0 = time.time()
X_tsne_3d = TSNE(n_components=3, perplexity=BEST_PERPLEXITY, init="pca",
                 learning_rate="auto", random_state=RANDOM_STATE).fit_transform(X)
print(f" готово за {time.time() - t0:.1f} c")

fig = plt.figure(figsize=(7.5, 6.5))
ax = fig.add_subplot(111, projection="3d")
scatter_3d(ax, X_tsne_3d, y, f"t-SNE (3D), perplexity={BEST_PERPLEXITY}",
           "t-SNE 1", "t-SNE 2", "t-SNE 3")
plt.tight_layout()
plt.savefig("tsne_3d.png", dpi=150)
plt.show()
plt.close()

fig, axes = plt.subplots(1, 3, figsize=(16, 5.5))
scatter_2d(axes[0], X_pca_2d, y, "PCA", "PC1", "PC2")

```

```

scatter_2d(axes[1], X_ae_2d, y, "Автоэнкодер")
scatter_2d(axes[2], X_tsne_2d, y, f"t-SNE (perplexity={BEST_PERPLEXITY})",
          "t-SNE 1", "t-SNE 2")
plt.suptitle("Сравнение методов снижения размерности (2D)", fontsize=13)
plt.tight_layout()
plt.savefig("comparison_all_methods_2d.png", dpi=150)
plt.show()
plt.close()

def class_overlap_estimate(data2d, labels):
    nn = NearestNeighbors(n_neighbors=16).fit(data2d)
    _, idx = nn.kneighbors(data2d)
    idx = idx[:, 1:]
    neighbor_labels = labels[idx]
    own_label = labels[:, None]
    frac_same = (neighbor_labels == own_label).mean(axis=1)
    mismatched = (frac_same < 0.5).mean() * 100
    return mismatched

mism_pca = class_overlap_estimate(X_pca_2d, y)
mism_ae = class_overlap_estimate(X_ae_2d, y)
mism_tsne = class_overlap_estimate(X_tsne_2d, y)

```

Результат программы:

Пропуски по признакам (до замещения):

```

Area          0
Perimeter     0
Major_Axis_Length  0
Minor_Axis_Length  0
Eccentricity    0
Convex_Area    0
Extent        0

```

Размер выборки после замещения пропусков: 3810 объектов, 7 признаков

```

Class
Osmancik  2180
Cammeo    1630

```

PCA: собственные значения ковариационной матрицы (по убыванию)

```

lambda_1 = 4.5802  доля объясненной дисперсии = 65.41%
lambda_2 = 1.5002  доля объясненной дисперсии = 21.43%
lambda_3 = 0.9010  доля объясненной дисперсии = 12.87%
lambda_4 = 0.0118  доля объясненной дисперсии = 0.17%
lambda_5 = 0.0062  доля объясненной дисперсии = 0.09%
lambda_6 = 0.0021  доля объясненной дисперсии = 0.03%
lambda_7 = 0.0004  доля объясненной дисперсии = 0.01%

```

Потери информативности при сведении к 2 компонентам: 13.16%

Потери информативности при сведении к 3 компонентам: 0.29%

Проверка через sklearn:

```

2 компоненты: [0.6541 0.2143] сумма = 86.84 %
3 компоненты: [0.6541 0.2143 0.1287] сумма = 99.71 %

```

АВТОЭНКОДЕР: обучение модели с 2 нейронами в среднем слое

эпоха 100/600 MSE(восст.) = 0.08933
эпоха 200/600 MSE(восст.) = 0.05764
эпоха 300/600 MSE(восст.) = 0.05132
эпоха 400/600 MSE(восст.) = 0.04678
эпоха 500/600 MSE(восст.) = 0.04611
эпоха 600/600 MSE(восст.) = 0.04706
обучение заняло 20.1 с, итоговая MSE восстановления = 0.04706

АВТОЭНКODER: обучение модели с 3 нейронами в среднем слое

эпоха 100/600 MSE(восст.) = 0.00246
эпоха 200/600 MSE(восст.) = 0.00180
эпоха 300/600 MSE(восст.) = 0.00161
эпоха 400/600 MSE(восст.) = 0.00159
эпоха 500/600 MSE(восст.) = 0.00151
эпоха 600/600 MSE(восст.) = 0.00150
обучение заняло 19.0 с, итоговая MSE восстановления = 0.00150

t-SNE: подбор perplexity (2 компоненты)

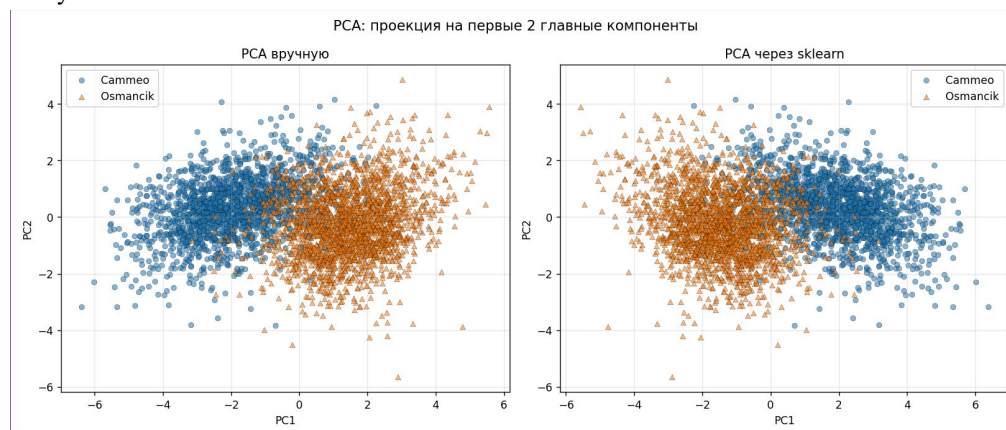
perplexity=20 (17.6 с)
perplexity=30 (19.7 с)
perplexity=40 (22.3 с)
perplexity=50 (20.2 с)
perplexity=60 (23.0 с)

Выбранное значение perplexity для итоговой визуализации: 30

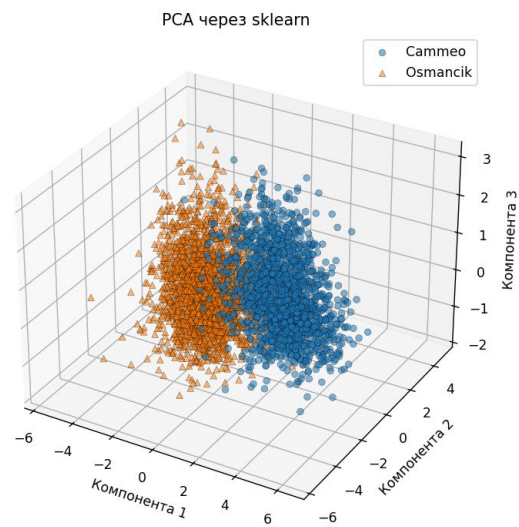
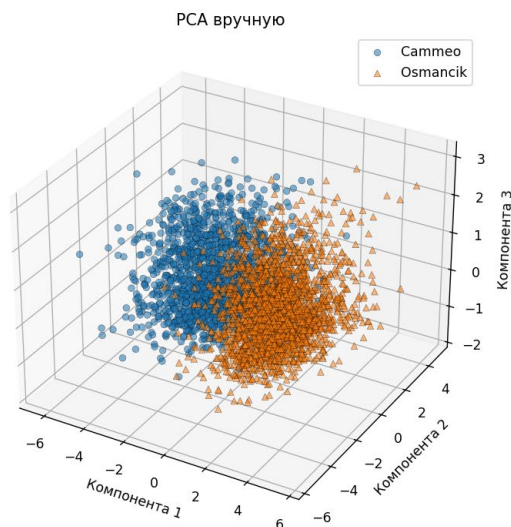
t-SNE: построение 3-компонентной проекции (perplexity=30)

готово за 56.9 с

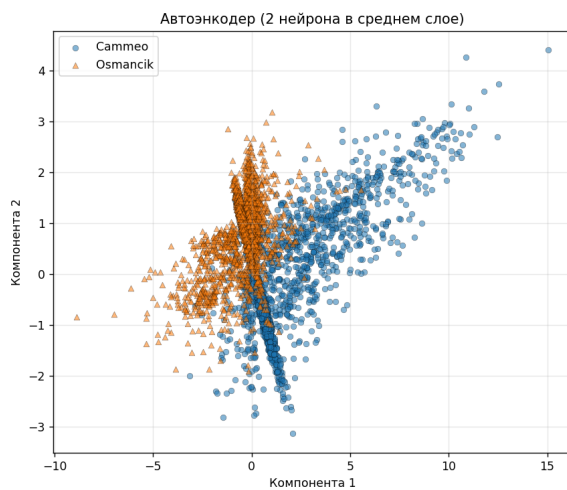
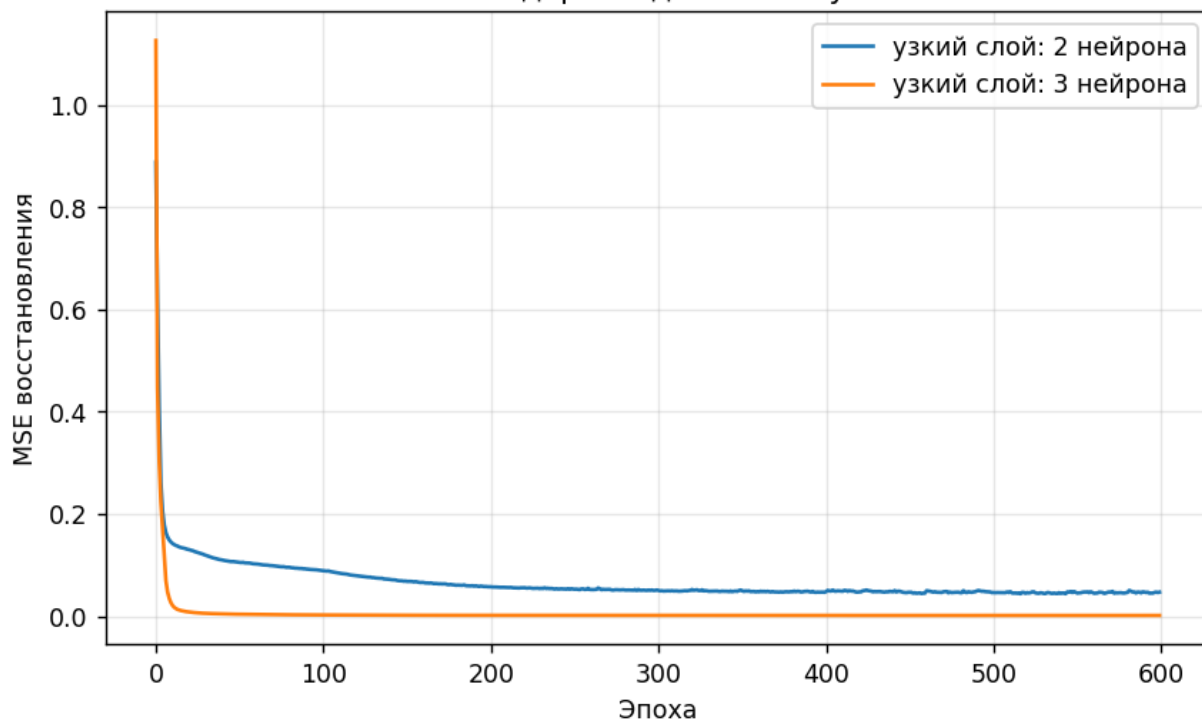
Полученные главные компоненты:



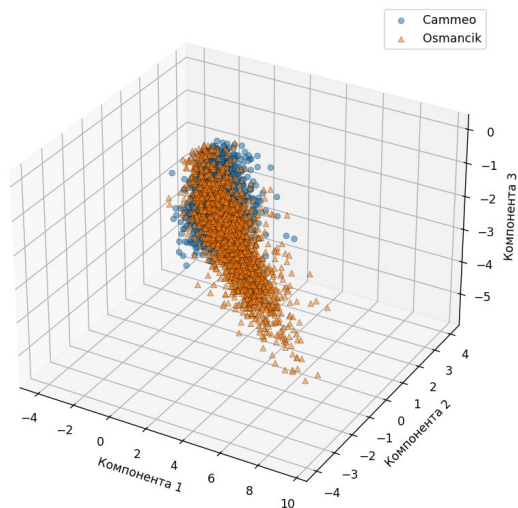
PCA: проекция на первые 3 главные компоненты



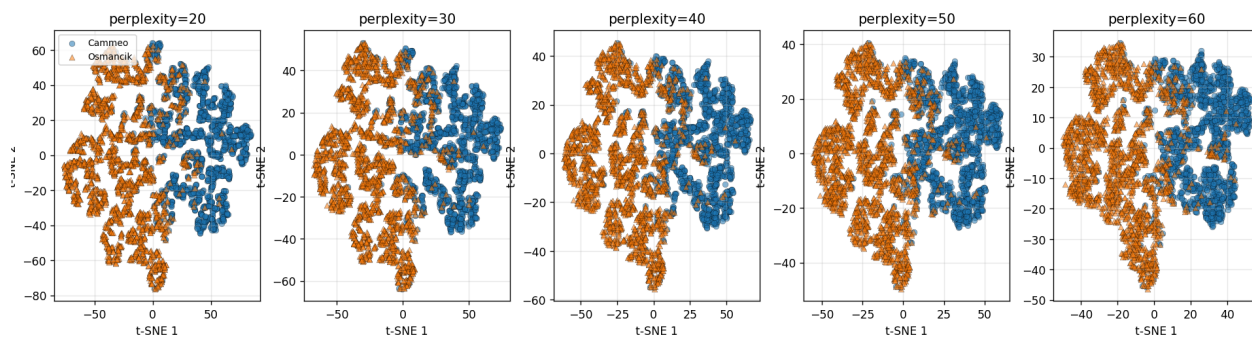
Автоэнкодер: сходимость обучения



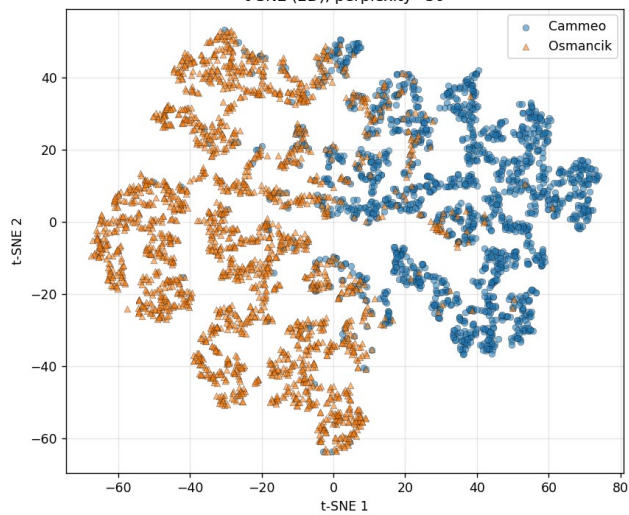
Автоэнкодер (3 нейрона в среднем слое)



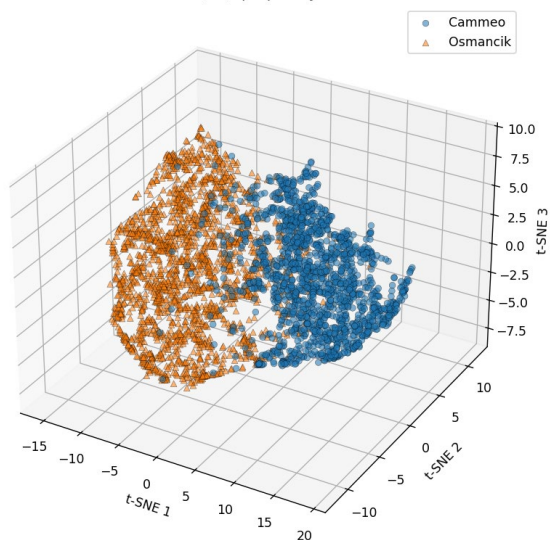
t-SNE (2D): сравнение значений perplexity



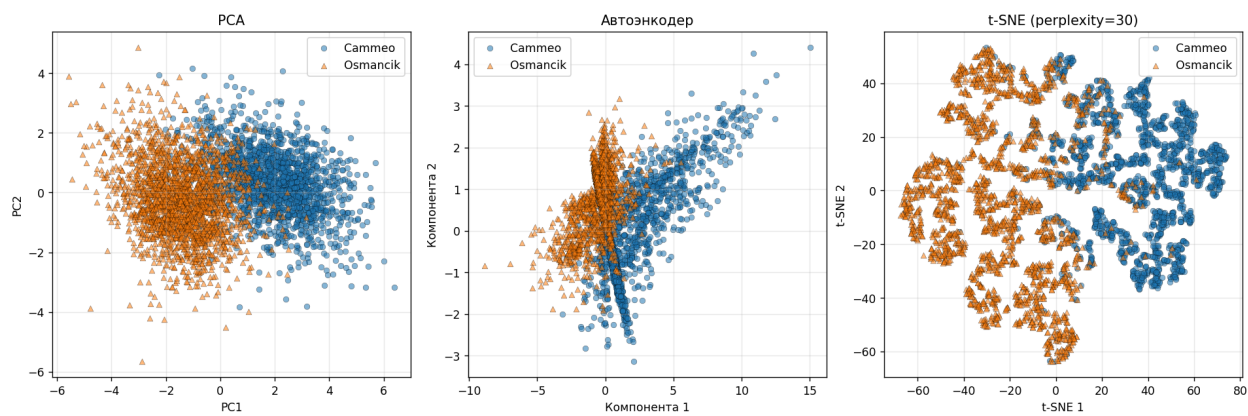
t-SNE (2D), perplexity=30



t-SNE (3D), perplexity=30



Сравнение методов снижения размерности (2D)



Вывод: Прodelав данную лабораторную работу я научился применять автоэнкодеры для осуществления визуализации данных и их анализа